FreshPay

Table of Contents

introduction

This addendum provides specific event calls to be issued against the PayDRC Processor. It outlines the methods and policies for interacting with the FreshPay Payment Switch for send money and payment transactions.

The PayDRC Processor interfaces with the FreshPay Switch, which also interfaces with gateways and systems managing a customer’s wallet. This wallet, in the case of mobile network operators, is the "Mobile Money Platform", or in the case of banks, the "cards platforms". It is physically separated from the payment gateway, sometimes referred to as a "Broker" or an "Orchestration Layer", providing the benefit of a simplified interface and abstraction of both function and security.

This document provides detailed usage of the PayDRC processor API, allowing developers and third-party plugins to send requests to the FreshPay switch engine to debit a FreshPay customer account (C2B) or credit a FreshPay customer account (B2C).

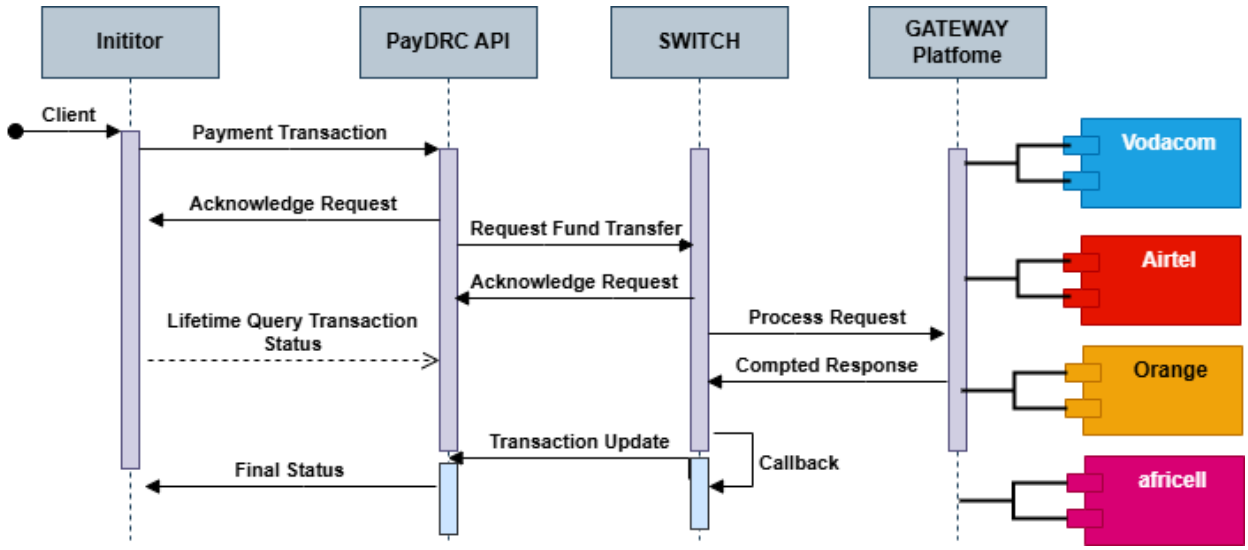
1. Transaction Handling

Communication with the PayDRC processor API follows an asynchronous communication model. In this model, the initiator sends a request to the API, which returns a response that is not the final status of the transaction. The initial response from the PayDRC API is merely an acknowledgment to indicate that the money transfer request has been received.

However, to obtain the final status of the transaction, the initiator must perform a check using an appropriate search action via the same API, but with a slightly different parameter format. This allows the initiator to retrieve the updated status of the transaction.

Additionally, in addition to transaction checking, final status notifications can also be sent to the initiator via a callback mechanism. When making a transaction request, you can specify a callback URL in the request parameters. Once the final status of the transaction is available, PayDRC will send a notification to this callback URL with the transaction details, including the final status.

2. Basic Flow



3. PayIn And PayOut API

Merchants can interact with the processor to either place a request for PayIn (C2B) or PayOut (B2C), or to request the status of a particular transaction.

3.1. Parameters:

- merchant_id (string, required): ***Your FreshPay merchant ID***.
- merchant_secrete (string, required): ***Your FreshPay merchant secret key***.
- firstname (string, required): ***Your FreshPay first name***.
- lastname (string, required): ***Your FreshPay last name***.
- email (string, required): ***Your FreshPay email address***.
- amount (string, required): The transaction amount.
- currency (string, required): The currency of the transaction.
- action (string, required): The action to perform (debit for payin and credit for payout).
- customer_number (string, required): The customer's phone number.
- reference (string, required): A unique reference for the transaction.
- method (string, required): The payment method to use (e.g., airtel, mtn, etc.).
- callback_url (string, optional): The URL where FreshPay will send transaction notifications.

3.2. PayIn (C2B) API

(Customer-to-Business Payment Collection via Mobile Money)

This API enables merchants to securely collect payments directly from a customer's mobile money wallet, with transaction authorization via SMS/Push notification.

Key Features

- ***Push Payment Authorization***:
  - Customer initiates payment on merchant platform
  - Instant SMS/Push notification sent by mobile operator (Airtel, Orange, etc.) for PIN confirmation
  - Funds only deducted after customer validation
- ***Real-Time Notifications***:
  - Instant transaction status updates (success/failed) via callback_url
  - Seamless integration with Check Status API for reconciliation
- ***Multi-Network Support***:
  - Airtel Money, Orange Money, M-Pesa and Afrimoney.
  - Multi-currency support (CDF, USD)

Example Use Cases

1. Bill payments (utilities, taxes, insurance)
2. E-commerce checkouts
3. Service subscriptions (SaaS, digital content)
4. In-store POS payments

FreshPay

TEST Endpoint:

`POST https://api.gofreshpay.com/api/v1/gateway`

PROD Endpoint:

`POST https://paydrc.gofreshbakery.net/api/v5/`

Example Request:

```
{
  "merchant_id": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "merchant_secrete": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "amount": "100",
  "currency": "CDF",
  "action": "debit",
  "customer_number": "0972148867",
  "firstname": "John",
  "lastname": "Doe",
  "e-mail": "john.doe@example.com",
  "reference": "testfp09",
  "method": "airtel",
  "callback_url": "https://yourdomain.com/callback"
}
```

Example Response:

```
{
  "Amount": 100,
  "Comment": "Transaction Received Successfully",
  "Created_At": "2024-03-08 08:08:51.533410",
  "Currency": "CDF",
  "Customer_Number": "972148867",
  "Reference": "testfp09",
  "Status": "Success",
  "Transaction_id": "PDABXkT03IfD08M9PfR24Ci4",
  "Updated_At": "2024-03-08 08:08:51.533410"
}
```

Note: The `"Status": "Success"` in this response only confirms FreshPay received the request. The actual transaction status (**Submitted/Pending/Failed/Successful**) is available via **Check Status API** or **Callback**.

3.3. PayOut (B2C) API

(Business-to-Customer Instant Mobile Money Transfers)

This API enables merchants to send instant payments directly to customers' mobile wallets without requiring recipient PIN validation. Funds are credited immediately upon successful processing.

Key Features

⚡ **Instant Transfers**:

- Direct credit to recipient's mobile money account
- No recipient action required (PIN-less transactions)

📊 **Comprehensive Tracking**:

- Real-time status updates via callback URLs
- Automated reconciliation with unique transaction references

🌍 **Multi-Operator Support**:

- Airtel Money | Orange Money | M-Pesa | Afrimoney
- Multi-currency support (CDF, USD)

Typical Use Cases

1. Salary payments to employees/contractors
2. Cashback and refund processing
3. Vendor/supplier payments
4. Loan disbursements
5. Commission payouts

TEST Endpoint:

`POST https://api.gofreshpay.com/api/v1/gateway`

PROD Endpoint:

`POST https://paydrc.gofreshbakery.net/api/v5/`

Example Request:

```
{
  "merchant_id": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "merchant_secrete": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "amount": "100",
  "currency": "CDF",
  "action": "credit",
  "customer_number": "0972148867",
  "firstname": "Jane",
  "lastname": "Doe",
  "e-mail": "jane.doe@example.com",
  "reference": "testfp09",
  "method": "airtel",
  "callback_url": "https://yourdomain.com/callback"
}
```

Example Response:

```
{
  "Amount": 100,
  "Comment": "Transaction Received Successfully",
  "Created_At": "2024-03-08 08:08:51.533410",
  "Currency": "CDF",
  "Customer_Number": "972148867",
  "Reference": "testfp09",
  "Status": "Success",
  "Transaction_id": "PDABXkT03IfD08M9PfR24Ci4",
  "Updated_At": "2024-03-08 08:08:51.533410"
}
```

Note: The `"Status": "Success"` in this response only confirms FreshPay received the request. The actual transaction status (**Submitted/Pending/Failed/Successful**) is available via **Check Status API** or **Callback**.

FreshPay

4. Check Status API

This API enables merchants to track the real-time status of their payment transactions and monitor their progress.

Key Features:

- Real-Time Tracking: Check the current status of transactions (e.g., pending, success, failed).
- Comprehensive Monitoring: Retrieve detailed transaction updates, including timestamps and error messages (if any).
- Seamless Integration: Simple API calls to verify transactions using a unique reference ID.

Example Use Case:

After initiating a payment, merchants can use the verify endpoint to confirm whether the transaction was completed successfully or if further action is needed.

Why It Matters:

- Transparency: Eliminates uncertainty by providing clear transaction states.
- Efficiency: Reduces manual checks with automated status updates.
- Error Handling: Immediate feedback for troubleshooting failed transactions.

TEST Endpoint:

```
POST https://api.gofreshpay.com/api/v1/gateway
```

PROD Endpoint:

```
POST https://paydrc.gofreshbakery.net/api/v5/
```

Example Request:


```
{
  "merchant_id": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "merchant_secrete": "xxxxxxxxxxxxxxxxxxxxxxxxx",
  "action": "verify",
  "reference": "PDYjZJL10X8J26VcQmM23bHV"
}
```


Example Response:


```
{
  "Action": "debit",
  "Amount": 100.0,
  "Comment": "Transaction Found",
  "Created_at": "2024-03-08 08:05:37",
  "Currency": "CDF",
  "Customer_Details": "972148867",
  "Financial_Institution_id": "null",
  "Method": "airtel",
  "Reference": "testfp09",
  "Status": "Success",
  "Trans_Status": "Failed",
  "Trans_Status_Description": "La reference de la transaction est invalide, v
euillez reesayez ou contactez le service client au 1213",
  "Transaction_id": "PD9pLLM03QZJ08X7fXM242x6",
  "Updated_at": "2024-03-08 08:51:30"
}
```

5. Callback Implementation

Introduction

FreshPay provides real-time transaction status updates through **secure HTTP POST callbacks** to the merchant's specified callback_url. These automated notifications ensure instant awareness of payment outcomes (success, failed, or pending) without manual polling.

Security Features:

- **AES-256-CBC Encryption**: Protects sensitive data during transmission.
- **HMAC-SHA256 Signature**: Verifies message authenticity and integrity.
- **IP Whitelisting**: Restricts callback access to trusted FreshPay servers.

1. Security and Compliance

1.1. HMAC-SHA256 Signature

To guarantee the integrity of incoming data, FreshPay Congo uses an **HMAC-SHA256** signature. This ensures that the message has not been altered and originates from a trusted source.

How it works:

- The sender generates a signature using the **HMAC-SHA256** algorithm with a secret key.
- The recipient recalculates the signature using the same method and compares it to the received signature.
- If both signatures match, the message is authenticated.

1.2. AES-256-CBC Encryption

To ensure that only the intended recipient can read the data, **AES-256-CBC encryption** is used.

How it works:

- The message is encrypted using an **AES key** and an **initialization vector (IV)**.
- The recipient decrypts the message using the same **AES key**.

Even if an attacker intercepts the message, they cannot read it without the secret key.

1.3. IP Whitelisting

FreshPay Congo requires that clients **whitelist our IP addresses**. This ensures that only authorized systems can send or receive callbacks.

Security Benefits:

- Prevents **unauthorized requests** from external sources.
- Reduces the risk of **DDoS attacks** or **fake transactions**.
- Ensures callbacks are only processed by trusted partners.

2. Callback Details

2.1. Request Format

Each payment notification sent by FreshPay includes:

- **HTTP Method:** POST
- **Required Headers:**

```
✓ Content-Type: application/json
✓ X-Signature: <HMAC_SHA256_SIGNATURE>
```

- **Body Format:** (JSON)

  Example Request:

```
{
  "data": "<ENCRYPTED_PAYLOAD>"
}
```

FreshPay

2.2. Expected Response Format

Your server must respond with either a success or an error message based on the validation of the request.

Successful Response (200 OK)

```
{
  "status": "Callback received successfully",
  "data": { ... }
}
```

Error Responses:

```
400 Bad Request → Invalid encryption
401 Unauthorized → Invalid signature
```

3. Implementation in Different Languages

- Below are secure implementations in **Python, PHP, and Node.js** for processing the callback safely.

Python Implementation

```
import json
import base64
import hashlib
import hmac
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
from flask import Flask, request, jsonify

SECRET_KEY = b'xxxxxxxxxxxxxxxx'  # 16-byte AES key
HMAC_KEY = b'xxxxxxxxxxxxxxxx'

app = Flask(__name__)

def decrypt_data(encrypted_data):
    cipher = AES.new(SECRET_KEY, AES.MODE_CBC, iv=SECRET_KEY)
    decrypted_bytes = unpad(cipher.decrypt(base64.b64decode(encrypted_data)),
AES.block_size)
    return json.loads(decrypted_bytes.decode())

def verify_signature(encrypted_message, received_signature):
```

```
    calculated_signature = hmac.new(HMAC_KEY, encrypted_message.encode(), has
hlib.sha256).hexdigest()
    return hmac.compare_digest(calculated_signature, received_signature)

@app.route('/callback', methods=['POST'])
def callback_handler():
    data = request.json.get("data")
    received_signature = request.headers.get("X-Signature")

    if not received_signature:
        return jsonify({"error": "Signature missing"}), 400

    if verify_signature(data, received_signature):
        try:
            decrypted_data = decrypt_data(data)
            return jsonify({"status": "Callback received", "data": decrypted_
data}), 200
        except Exception:
            return jsonify({"error": "Invalid encryption"}), 400
    else:
        return jsonify({"error": "Invalid signature"}), 401
```

FreshPay

PHP Implementation

```
<?php
define('SECRET_KEY', 'xxxxxxxxxxxxxxx');
define('HMAC_KEY', 'xxxxxxxxxxxxxxx');

function decrypt_data($encrypted_data) {
    $decoded_data = base64_decode($encrypted_data);
    $iv = SECRET_KEY;
    $decrypted = openssl_decrypt($decoded_data, 'AES-128-CBC', SECRET_KEY, OP
ENSSL_RAW_DATA, $iv);
    return json_decode($decrypted, true);
}

function verify_signature($encrypted_message, $received_signature) {
    $calculated_signature = hash_hmac('sha256', $encrypted_message, HMAC_KEY)
;
    return hash_equals($calculated_signature, $received_signature);
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $data = json_decode(file_get_contents('php://input'), true)['data'];
    $received_signature = $_SERVER['HTTP_X_SIGNATURE'] ?? '';

    if (!$received_signature) {
        echo json_encode(["error" => "Signature missing"]);
        http_response_code(400);
        exit;
    }

    if (verify_signature($data, $received_signature)) {
        try {
            $decrypted_data = decrypt_data($data);
            echo json_encode(["status" => "Callback received", "data" => $dec
rypted_data]);
            http_response_code(200);
        } catch (Exception $e) {
            echo json_encode(["error" => "Invalid encryption"]);
            http_response_code(400);
        }
    } else {
        echo json_encode(["error" => "Invalid signature"]);
        http_response_code(401);
    }
}?>
```

FreshPay

Node.js Implementation (Express.js)

```
const express = require('express');
const crypto = require('crypto');
const base64 = require('base-64');
const bodyParser = require('body-parser');

const app = express();
app.use(bodyParser.json());

const SECRET_KEY = Buffer.from('xxxxxxxxxxxxxxx'); // 16-byte AES key
const HMAC_KEY = 'xxxxxxxxxxxxxxx';

// Function to decrypt AES-256-CBC data
function decryptData(encryptedData) {
    const decipher = crypto.createDecipheriv('aes-128-cbc', SECRET_KEY, SECRE
T_KEY);
    let decrypted = decipher.update(base64.decode(encryptedData), 'base64', '
utf8');
    decrypted += decipher.final('utf8');
    return JSON.parse(decrypted);
}

// Function to verify HMAC-SHA256 signature
function verifySignature(encryptedMessage, receivedSignature) {
    const hmac = crypto.createHmac('sha256', HMAC_KEY);
    hmac.update(encryptedMessage);
    const calculatedSignature = hmac.digest('hex');
    return calculatedSignature === receivedSignature;
}

// Route to handle callback
app.post('/callback', (req, res) => {
    const data = req.body.data;
    const receivedSignature = req.headers['x-signature'];

    if (!receivedSignature) {
        return res.status(400).json({ error: "Signature missing" });
    }

    if (verifySignature(data, receivedSignature)) {
        try {
            const decryptedData = decryptData(data);
            return res.status(200).json({ status: "Callback received", data:
decryptedData });
```

```
        } catch (error) {
            return res.status(400).json({ error: "Invalid encryption" });
        }
    } else {
        return res.status(401).json({ error: "Invalid signature" });
    }
});

app.listen(3000, () => console.log('Callback server running on port 3000'));
```

Java Implementation (Spring Boot)

```
import org.springframework.web.bind.annotation.*;
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

@RestController
@RequestMapping("/callback")
public class CallbackController {

    private static final String SECRET_KEY = "xxxxxxxxxxxxxxx";
    private static final String HMAC_KEY = "xxxxxxxxxxxxxxx";

    // Function to verify HMAC signature
    private boolean verifySignature(String encryptedMessage, String receivedS
ignature) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        SecretKeySpec secretKeySpec = new SecretKeySpec(HMAC_KEY.getBytes(),
"HmacSHA256");
        mac.init(secretKeySpec);
        String calculatedSignature = bytesToHex(mac.doFinal(encryptedMessage.
getBytes()));
        return calculatedSignature.equals(receivedSignature);
    }

    // Function to decrypt AES-256-CBC data
    private String decryptData(String encryptedData) throws Exception {
        byte[] decodedData = Base64.getDecoder().decode(encryptedData);
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5PADDING");
```

```
        SecretKeySpec secretKeySpec = new SecretKeySpec(SECRET_KEY.getBytes()
, "AES");
        IvParameterSpec ivParameterSpec = new IvParameterSpec(SECRET_KEY.getB
ytes());
        cipher.init(Cipher.DECRYPT_MODE, secretKeySpec, ivParameterSpec);
        return new String(cipher.doFinal(decodedData));
    }

    // Helper function to convert bytes to hexadecimal
    private static String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }

    @PostMapping
    public ResponseEntity<?> handleCallback(@RequestBody Map<String, String>
requestBody, @RequestHeader("X-Signature") String receivedSignature) {
        String data = requestBody.get("data");

        if (receivedSignature == null) {
            return ResponseEntity.badRequest().body(Collections.singletonMap(
"error", "Signature missing"));
        }

        try {
            if (verifySignature(data, receivedSignature)) {
                String decryptedData = decryptData(data);
                return ResponseEntity.ok(Collections.singletonMap("status", "
Callback received successfully"));
            } else {
                return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Co
llections.singletonMap("error", "Invalid signature"));
            }
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(Collections.singletonMap(
"error", "Invalid encryption"));
        }
    }
}
```

6. Transaction Status Flow

1. **Initial Response (PayIn/PayOut APIs)**

   - ✓ The "Status": "Success" in PayIn/PayOut responses **only confirms receipt** of the request by FreshPay.

   - ✓ At this stage, the **actual transaction status** is Submitted (processing begins).

2. **Final Status**

   - ✓ Obtained via:
     - **Callback Notifications** *(Only final statuses: Failed or Successful)*
     - **Check Status API** *(All intermediate statuses: Submitted, Pending, Failed, Successful)*

6.1. Key Response Parameters

Parameter	Description
Trans_Status	**Final transaction status** (Failed/Successful in callbacks; includes intermediates in Check Status API).
Transaction_id	FreshPay's internal transaction reference.
Financial_Institution_id	Mobile money provider's transaction reference (e.g., MP250429.1824.B43262).
Reference	Merchant's original transaction reference.
Trans_Status_Description	Detailed failure reason (if applicable).

6.2. Example Callback/Check Status Response

```
{
  "Action": "debit",
  "Amount": 100.0,
  "Comment": "Transaction Found",
  "Created_at": "2024-03-08 08:05:37",
  "Currency": "CDF",
  "Customer_Details": "972148867",
  "Financial_Institution_id": "MP250429.1824.B43262",
  "Method": "airtel",
  "Reference": "testfp09",
  "Status": "Success",  // Confirms FreshPay received the request
  "Trans_Status": "Failed",  // Actual transaction outcome
  "Trans_Status_Description": "Invalid transaction reference",
  "Transaction_id": "PD9pLLM03QZJ08X7fXM242x6"
}
```

6.3. Key Differences: Callback vs. Check Status API

Feature	Callback	Check Status API
Trigger	Automatic (FreshPay-initiated)	Manual (Merchant-initiated)
Statuses	Only Failed/Successful	All (Submitted, Pending, Failed, Successful)
Use Case	Real-time final updates	On-demand status checks